ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 5 - Due date 02/17/26

Sylvia Hipp

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A05_Sp26.Rmd"). Then change "Student Name" on line 3 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Canvas.

R packages needed for this assignment: "readxl", "ggplot2", "forecast","tseries", and "Kendall". Install these packages, if you haven't done yet. Do not forget to load them before running your script, since they are NOT default packages.\

```
#Load/install required package here
library(forecast)
library(tseries)
library(ggplot2)
library(Kendall)
library(lubridate)
library(tidyverse)  #load this package so yon clean the data frame using pipes
library(readxl)
library(glue)
```

Consider the same data you used for A04 from the spreadsheet "Table_10.1_Renewable_Energy_Production_and_Consump
The data comes from the US Energy Information and Administration and corresponds to the December 2025 Monthly Energy Review.

```
#Importing data set - using readxl package
energy_data <- read_excel(
  path="./Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",
  skip = 12,
  sheet="Monthly Data",
  col_names=FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
#Now let's extract the column names from row 11 only
read_col_names <- read_excel(
  path="./Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",
  skip = 10,
  n_max = 1,
  sheet="Monthly Data",
  col_names=FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
colnames(energy_data) <- read_col_names
nobs <- nrow(energy_data)
nvar <- ncol(energy_data)

head(energy_data)
```

```
## # A tibble: 6 x 14
##   Month               `Wood Energy Production` `Biofuels Production`
##   <dttm>                                  <dbl> <chr>
## 1 1973-01-01 00:00:00                      130. Not Available
## 2 1973-02-01 00:00:00                      117. Not Available
## 3 1973-03-01 00:00:00                      130. Not Available
```

```
## 4 1973-04-01 00:00:00                    125. Not Available
## 5 1973-05-01 00:00:00                    130. Not Available
## 6 1973-06-01 00:00:00                    125. Not Available
## # i 11 more variables: ‘Total Biomass Energy Production‘ <dbl>,
## #   ‘Total Renewable Energy Production‘ <dbl>,
## #   ‘Hydroelectric Power Consumption‘ <dbl>,
## #   ‘Geothermal Energy Consumption‘ <dbl>, ‘Solar Energy Consumption‘ <chr>,
## #   ‘Wind Energy Consumption‘ <chr>, ‘Wood Energy Consumption‘ <dbl>,
## #   ‘Waste Energy Consumption‘ <dbl>, ‘Biofuels Consumption‘ <chr>,
## #   ‘Total Biomass Energy Consumption‘ <dbl>, ...
```

```r
energy_data <- energy_data %>% janitor::clean_names()  # clean up col names
```

Handling Missing Data

Q1

Using the original dataset, create a new data frame that includes only the following variables: **Date, Solar Energy Consumption and Wind Energy Consumption**. Check the class of columns, you will see that they are stored are characters instead of numbers. Because solar generation begins later in the sample, the early observations are recorded as "Not Available". Convert the data to numeric, the "Not Available" will became NAs.

You may either filter out the "Not Available" rows and then convert the column to numeric or convert first and then remove missing values using drop_na() (or na.omit()). If you are comfortable using pipes for data wrangling, please do so.

Important: Note that we dropping the missing observations instead of interpolating is because they only happen in the beginning of the series!

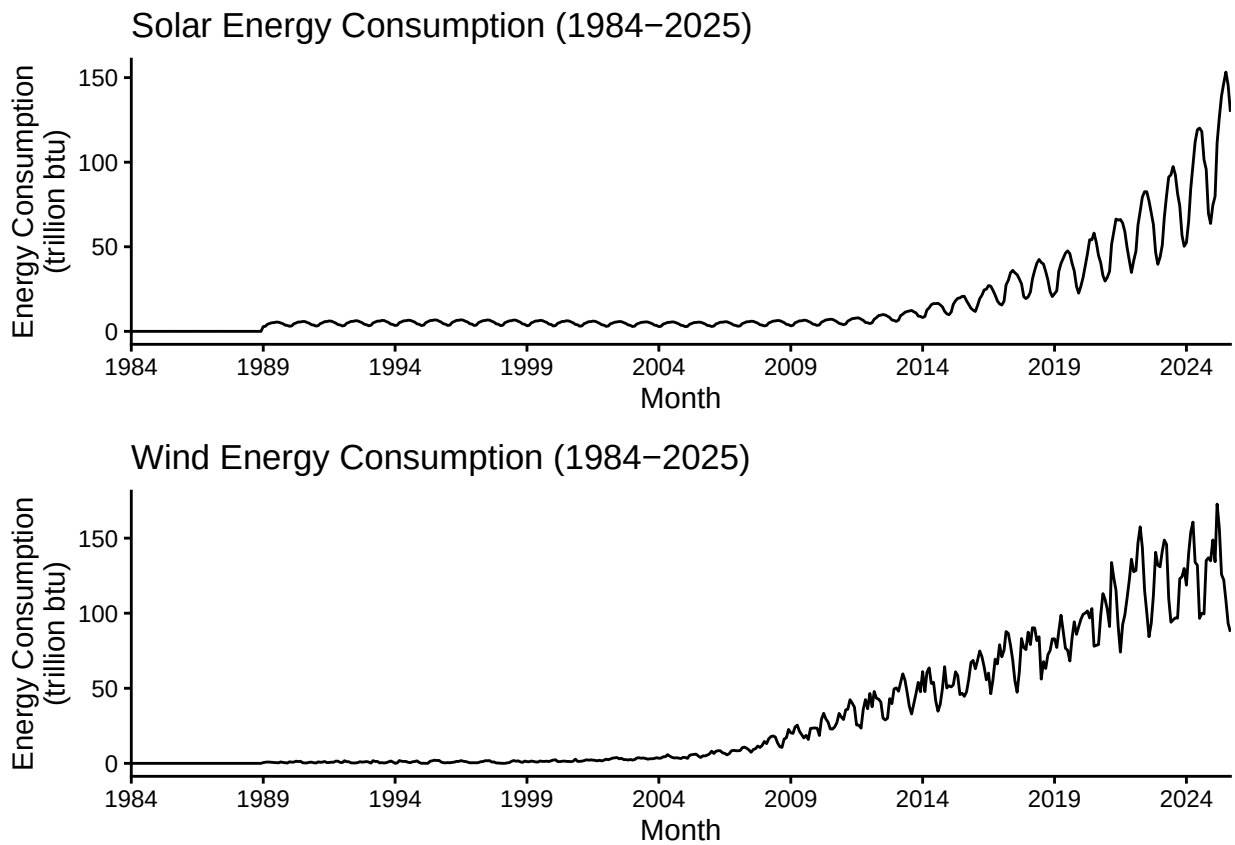
```r
renewables_df <- energy_data %>%
  select(month, solar_energy_consumption, wind_energy_consumption) %>%
  mutate(across(c(solar_energy_consumption:wind_energy_consumption), as.numeric)) %>%
  filter(!is.na(solar_energy_consumption),
         !is.na(wind_energy_consumption))
```

```
## Warning: There were 2 warnings in ‘mutate()‘.
## The first warning was:
## i In argument: ‘across(c(solar_energy_consumption:wind_energy_consumption),
##   as.numeric)‘.
## Caused by warning:
## ! NAs introduced by coercion
## i Run ‘dplyr::last_dplyr_warnings()‘ to see the 1 remaining warning.
```

Q2

Plot the Solar and Wind energy consumption over time using ggplot. Plot each series on a separate graph. No need to add legend. Add informative names to the y axis using `ylab()`. Explore the function `scale_x_date()` on ggplot and see if you can change the x axis to improve your plot. Hint: use `scale_x_date(date_breaks = "5 years", date_labels = "%Y")")`

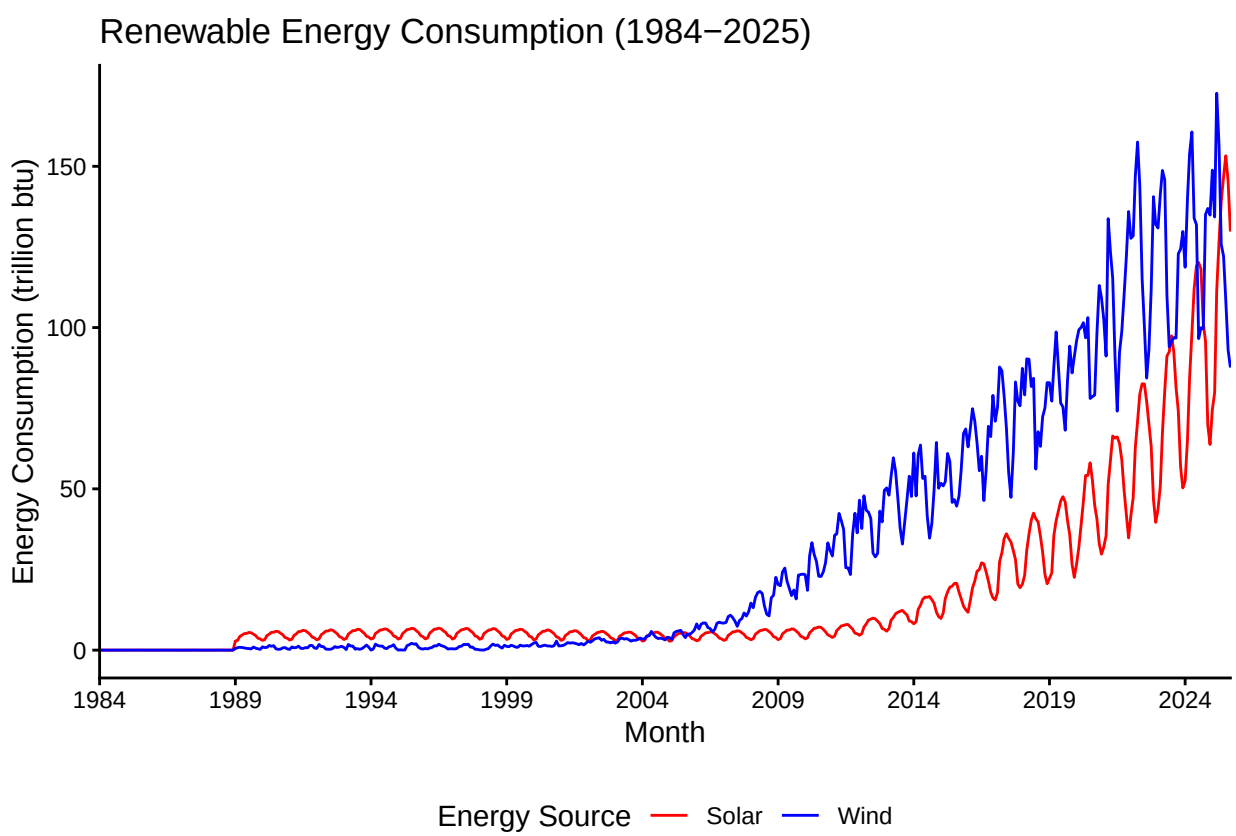
```r
plot_series <- function(series, label){
  ggplot(renewables_df, aes(x = month, y = series)) +
    geom_line() +
    theme_classic() +
    scale_x_date(date_breaks = "5 years", date_labels = "%Y",
                 expand = c(0,0)) +   # get rid of gap around edges
    labs(x = "Month",
         y = glue("Energy Consumption\n(trillion btu)"),
         title = glue("{label} Energy Consumption (1984-2025)"))
}

cowplot::plot_grid(
  plot_series(renewables_df$solar_energy_consumption, "Solar"),
  plot_series(renewables_df$wind_energy_consumption, "Wind"),
  nrow = 2)
```



Q3

Now plot both series in the same graph, also using ggplot(). Use function `scale_color_manual()` to manually add a legend to ggplot. Make the solar energy consumption red and wind energy consumption blue. Add informative name to the y axis using `ylab("Energy Consumption)`. And use function `scale_x_date()` to set x axis breaks every 5 years.

```r
renewables_df %>%
  pivot_longer(cols=c(solar_energy_consumption, wind_energy_consumption),
               names_to = "energy_source", values_to = "consumption") %>%
  mutate(energy_source = factor(energy_source,
                                levels = c("solar_energy_consumption", "wind_energy_consumption"),
                                labels = c("Solar", "Wind"))) %>%
  ggplot(aes(x = month, y = consumption, color = energy_source)) +
  geom_line() +
  scale_color_manual(values = c("red", "blue")) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y",
               expand = c(0,0)) +   # get rid of gap around edges
  theme_classic() +
  theme(legend.position = "bottom") +
  labs(x = "Month",
       y = "Energy Consumption (trillion btu)",
       color = "Energy Source",
       title = "Renewable Energy Consumption (1984-2025)")
```



Decomposing the time series

The stats package has a function called decompose(). This function only take time series object. As the name says the decompose function will decompose your time series into three components: trend, seasonal and random. This is similar to what we did in the previous script, but in a more automated way. The random component is the time series without seasonal and trend component.

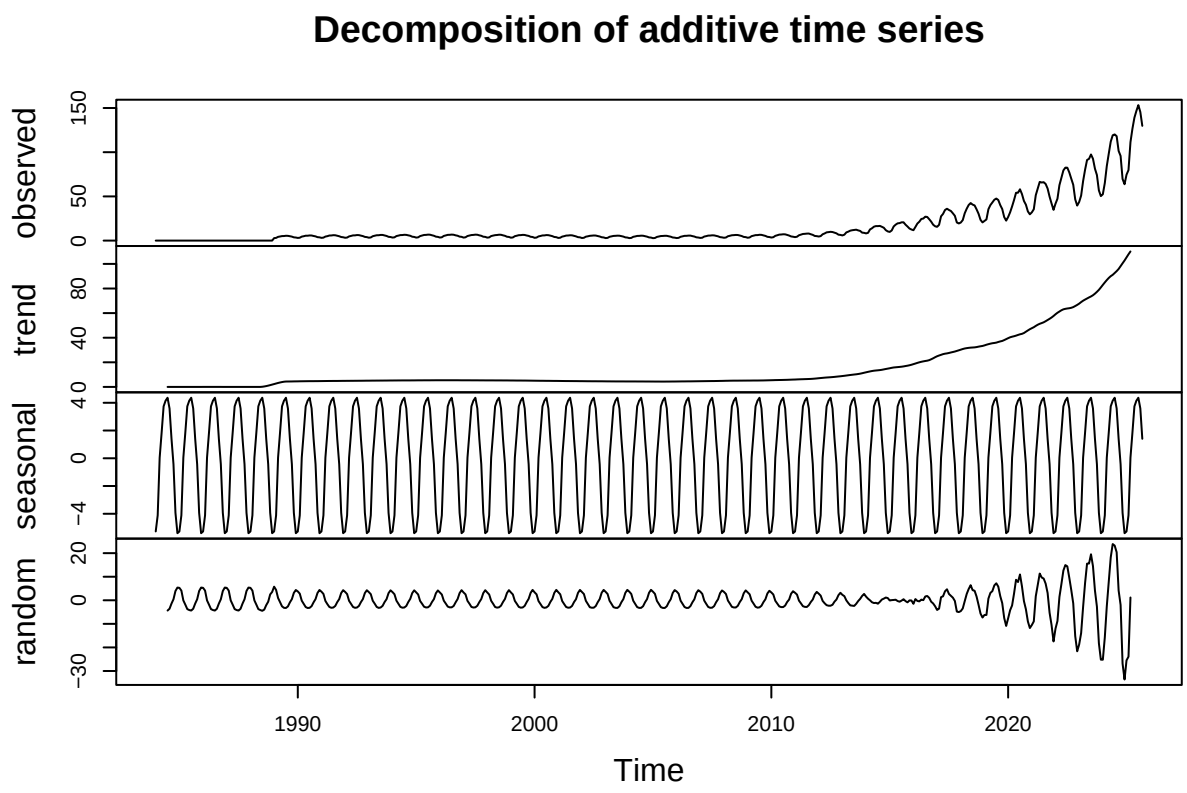
Additional info on `decompose()`.

1) You have two options: alternative and multiplicative. Multiplicative models exhibit a change in frequency over time.
2) The trend is not a straight line because it uses a moving average method to detect trend.
3) The seasonal component of the time series is found by subtracting the trend component from the original data then grouping the results by month and averaging them.
4) The random component, also referred to as the noise component, is composed of all the leftover signal which is not explained by the combination of the trend and seasonal component.

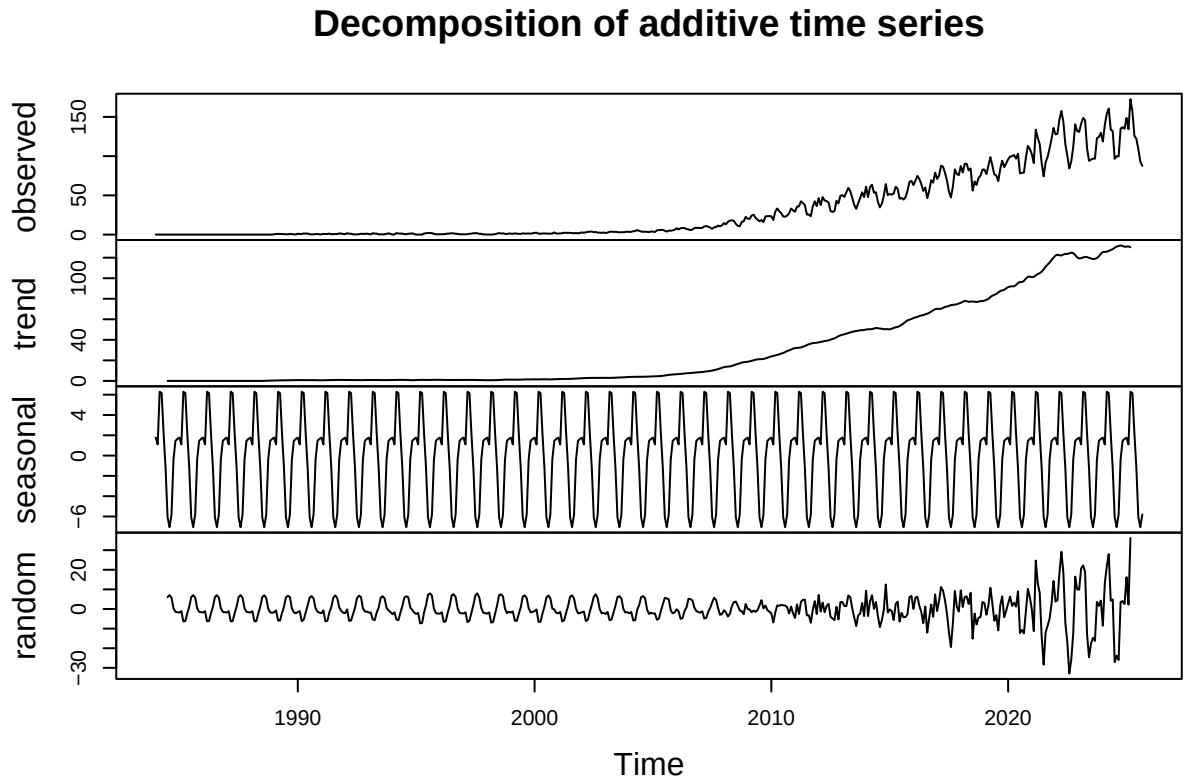
Q4

Transform wind and solar series into a time series object and apply the decompose function on them using the additive option, i.e., `decompose(ts_data, type = "additive")`. What can you say about the trend component? What about the random component? Does the random component look random? Or does it appear to still have some seasonality on it?

```
start <- first(renewables_df$month)

# create time series objects
solar_ts <- ts(renewables_df$solar_energy_consumption,
               start = c(year(start), month(start)), frequency = 12)
wind_ts <- ts(renewables_df$wind_energy_consumption,
              start = c(year(start), month(start)), frequency = 12)

# decompose time series
solar_ts_decomp <- decompose(solar_ts, type = "additive")
plot(solar_ts_decomp)
```




```r
wind_ts_decomp <- decompose(wind_ts, type = "additive")
plot(wind_ts_decomp)
```

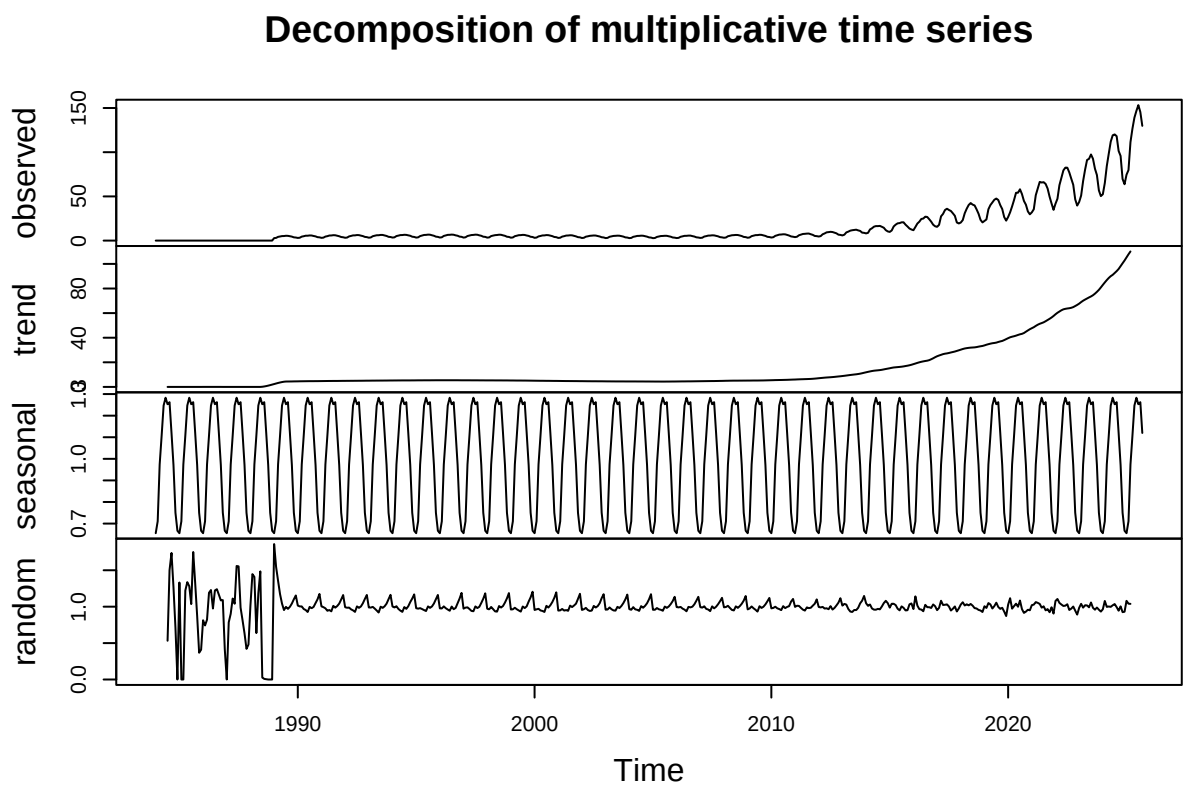


Answer: For the solar consumption series, there is a clear positive non-linear trend component. The random component appears to still have strong seasonality and trends over time (fluctuations become wider towards the end), which may suggest the presence of a stochastic process. We see the same patterns for the wind consumption series: there is a strong positive trend (that appears slightly more linear), and the random component retains some trend and seasonality.

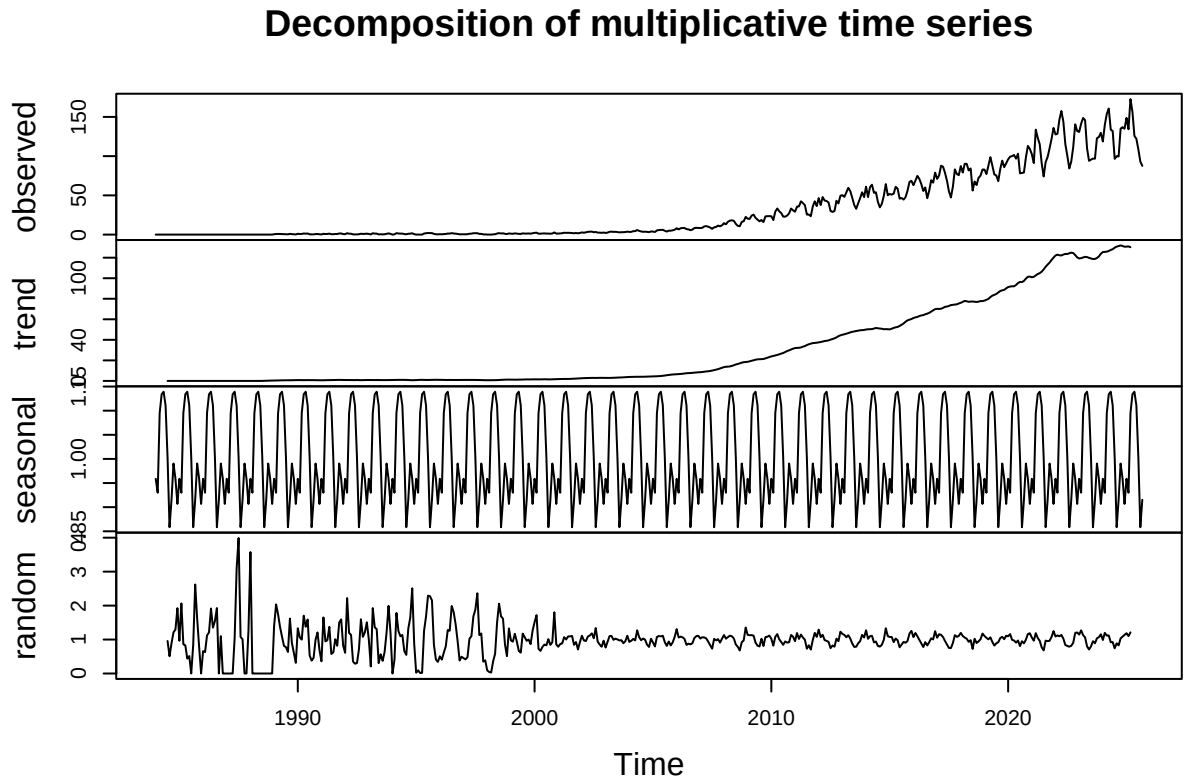
Q5

Use the decompose function again but now change the type of the seasonal component from additive to multiplicative. What happened to the random component this time?

```
solar_ts_decomp_mult <- decompose(solar_ts, type = "multiplicative")
plot(solar_ts_decomp_mult)
```




```r
wind_ts_decomp_mult <- decompose(wind_ts, type = "multiplicative")
plot(wind_ts_decomp_mult)
```



Using the multiplicative seasonal component did a better job at removing the seasonal component, where the random component of both decomposed series appears more like a truly random series. However, we still see some trend in both series where the random component is large at the start and becomes very small as time goes on. There also remains some seasonality, especially in the solar random series.

Q6

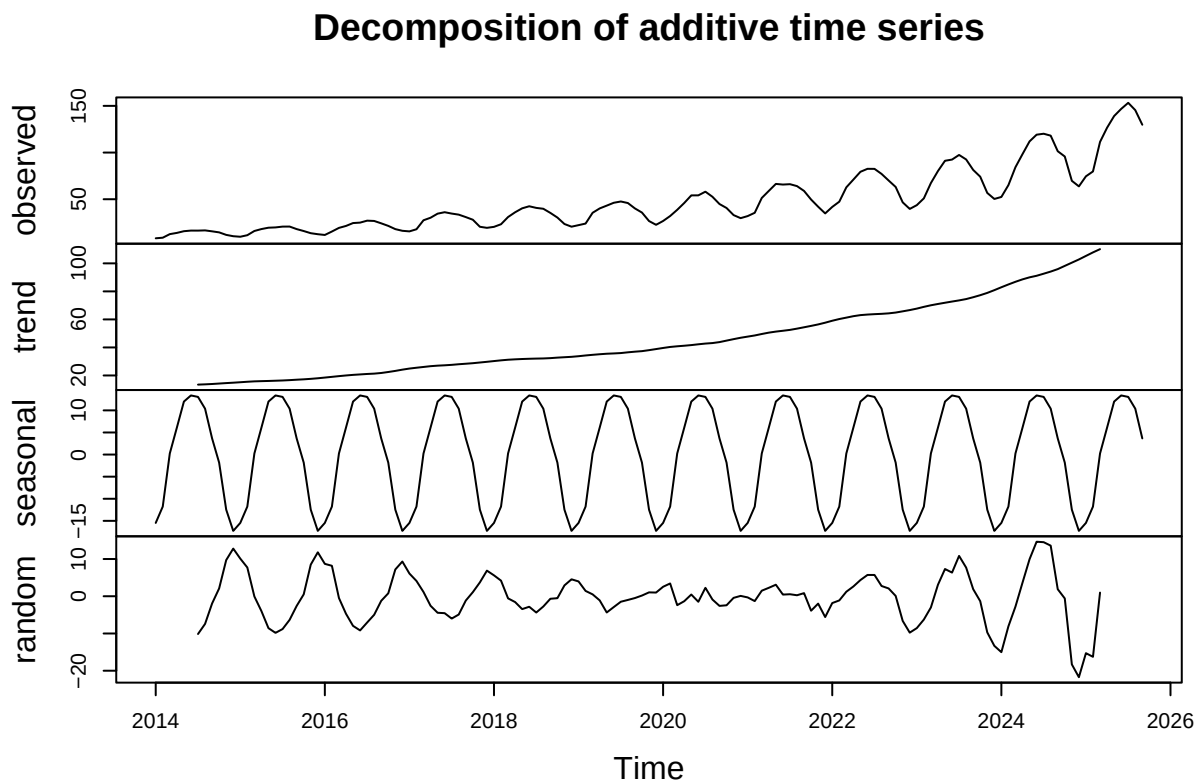
When fitting a model to this data, do you think you need all the historical data? Think about the data from 80s, 90s and early 20s. Are there any information from those years we might need to forecast the next six months of Solar and/or Wind consumption. Explain your response.

Answer: We do not see a significant growth in solar and wind energy consumption until the early 2000s, which was likely due to the introduction of cheaper and more efficient technologies. Prior to that point, renewable energy consumption did not have a strong trend and remained at low levels (<10 trillion Btu). It would not be necessary to include all data in the model because the series behavior from 1980s–2000s would not be a good reflection of expected trends in the next six months.

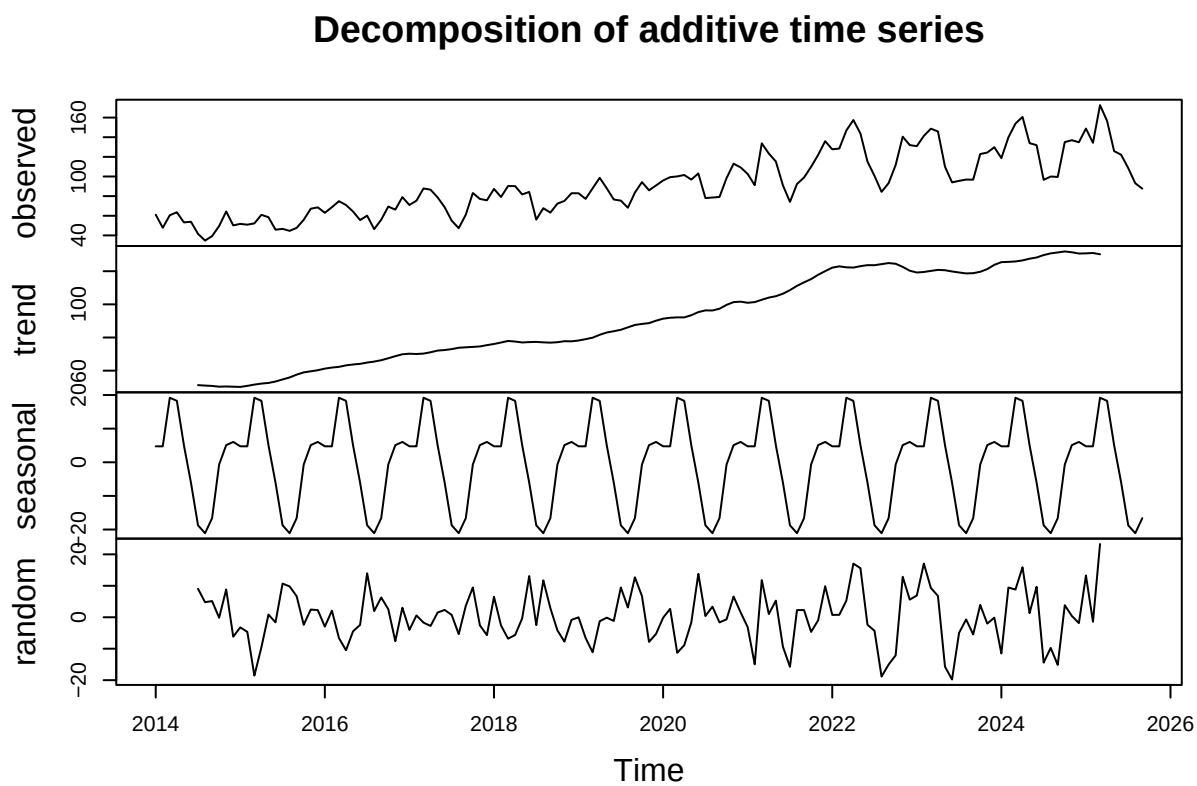
Q7

Create a new time series object where historical data starts on January 2014. Hint: use `filter()` function so that you don't need to point to row numbers, .i.e, `filter(xxxx, year(Date) >= 2014 )`. Apply the decompose function `type=additive` to this new time series. Comment on the results. Does the random component look random?

```r
start <- ymd("2014-01-01")

# solar time series post-2014
solar_ts_recent <- renewables_df %>%
  filter(month >= start) %>%
  pull(solar_energy_consumption) %>%
  ts(start = c(year(start), month(start)), frequency = 12)

solar_ts_recent_decomp <- decompose(solar_ts_recent, type = "additive")
plot(solar_ts_recent_decomp)
```




```r
# wind time series post-2014
wind_ts_recent <- renewables_df %>%
  filter(month >= start) %>%
  pull(wind_energy_consumption) %>%
  ts(start = c(year(start), month(start)), frequency = 12)

wind_ts_recent_decomp <- decompose(wind_ts_recent, type = "additive")
plot(wind_ts_recent_decomp)
```



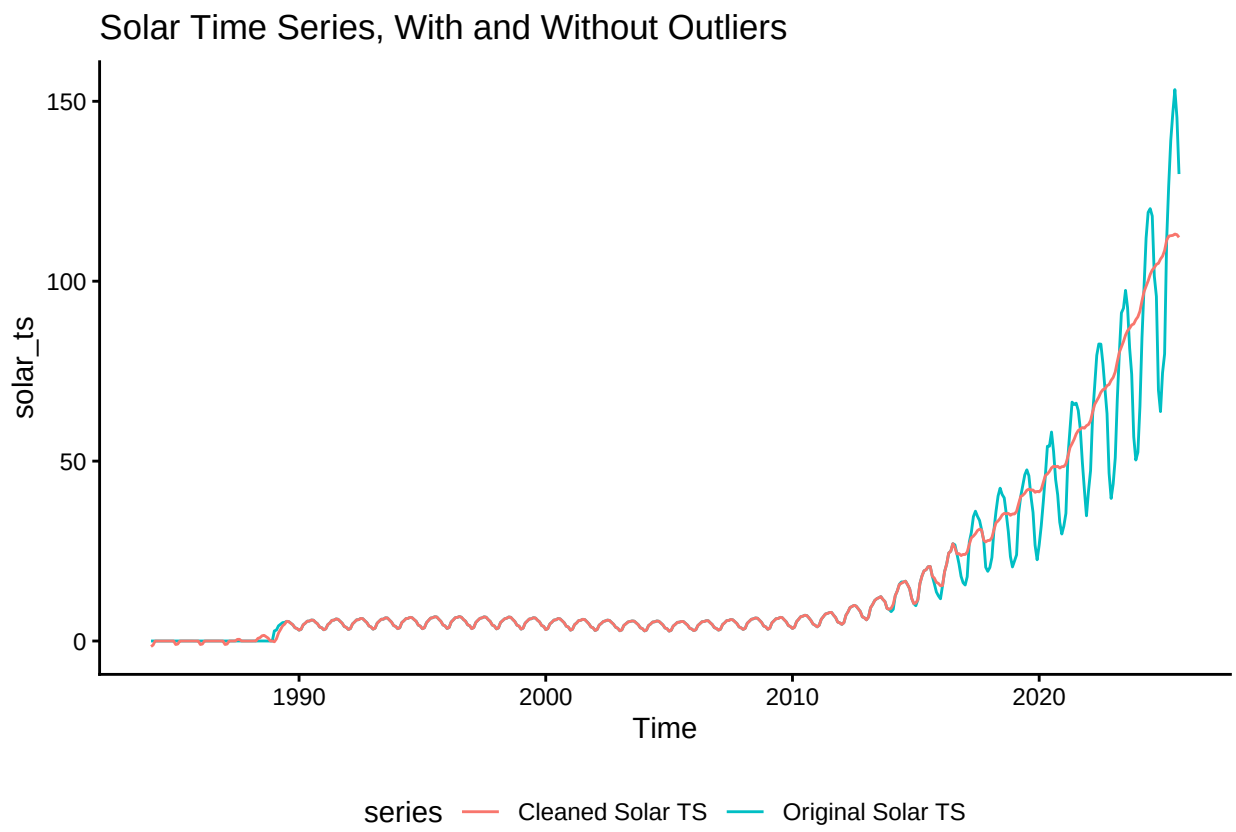
Answer: The random component of the solar series does not appear random, there is a clear seasonal component where the random term fluctuates over time and converges near 0 at the middle of the series. The wind series' random component appears a bit more random, though there does seem to be some seasonal component still.

Identify and Remove outliers

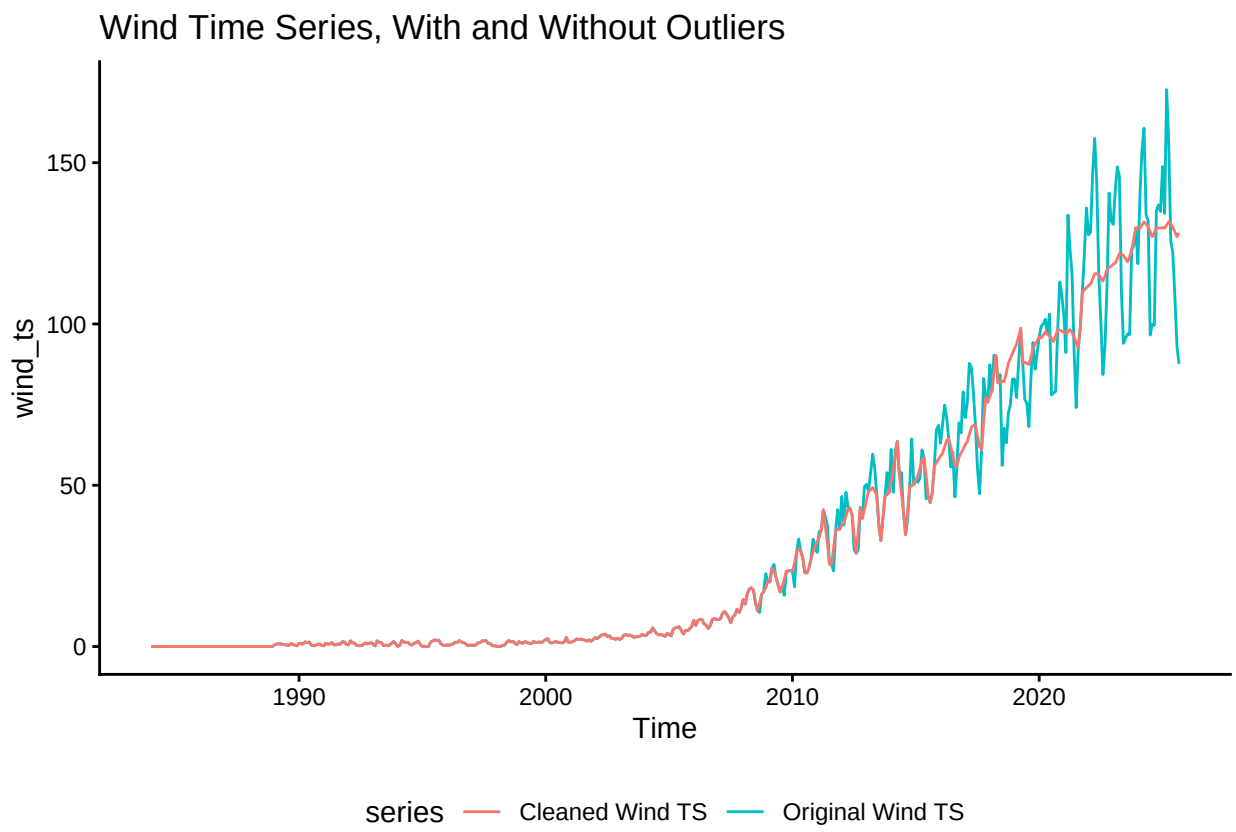
Q8

Apply the `tsclean()` to both time series object you created on Q4. Did the function removed any outliers from the series? Hint: Use `autoplot()` to check if there is difference between cleaned series and original series.

```
solar_ts_clean <- tsclean(solar_ts)
wind_ts_clean <- tsclean(wind_ts)

# plot clean and not cleaned time series
autoplot(solar_ts, series = "Original Solar TS") +
  autolayer(solar_ts_clean, series = "Cleaned Solar TS") +
  theme_classic() +
  theme(legend.position = "bottom") +
  labs(title = "Solar Time Series, With and Without Outliers")
```




```r
autoplot(wind_ts, series = "Original Wind TS") +
  autolayer(wind_ts_clean, series = "Cleaned Wind TS") +
  theme_classic() +
  theme(legend.position = "bottom") +
  labs(title = "Wind Time Series, With and Without Outliers")
```

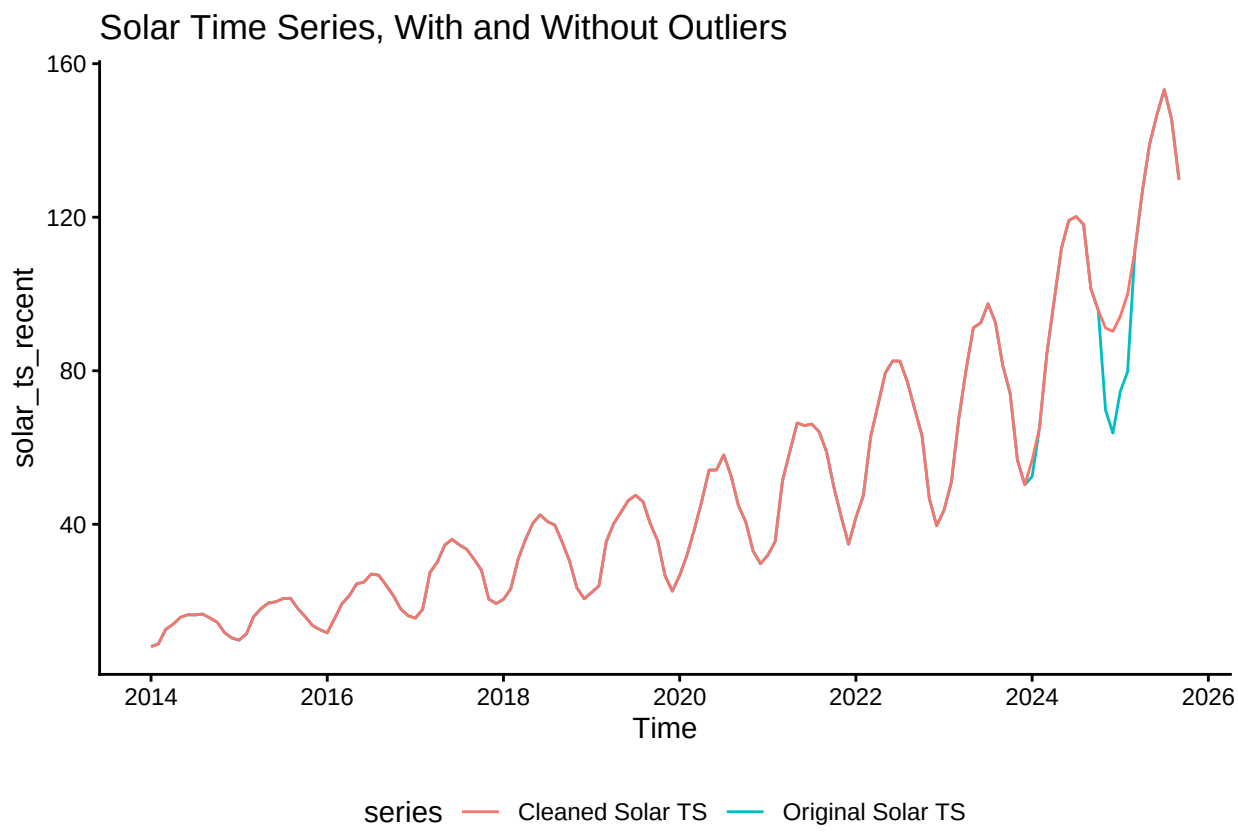


tsclean() seems to have removed outliers in both the solar and wind time series. For both series, it appears to have "smoothed" the overall trend from around 2015 onwards, so it removed all outliers caused by the wide seasonal fluctuations in renewable energy consumption.

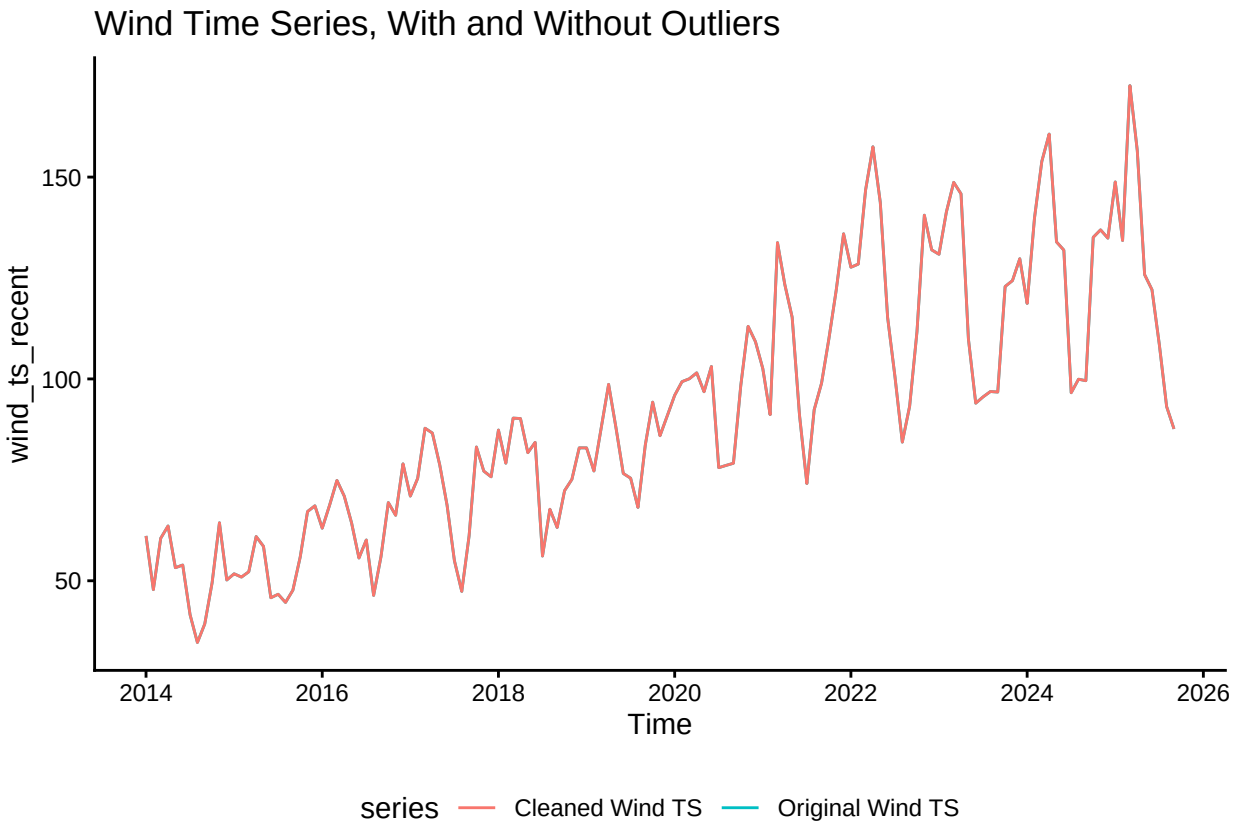
Q9

Redo number Q8 but now with the time series you created on Q7, i.e., the series starting in 2014. Using what `autoplot()` again what happened now? Did the function removed any outliers from the series?

```
solar_ts_clean_recent <- tsclean(solar_ts_recent)
wind_ts_clean_recent <- tsclean(wind_ts_recent)

# plot clean and not cleaned time series
autoplot(solar_ts_recent, series = "Original Solar TS") +
  autolayer(solar_ts_clean_recent, series = "Cleaned Solar TS") +
  theme_classic() +
  theme(legend.position = "bottom") +
  labs(title = "Solar Time Series, With and Without Outliers")
```




```r
autoplot(wind_ts_recent, series = "Original Wind TS") +
  autolayer(wind_ts_clean_recent, series = "Cleaned Wind TS") +
  theme_classic() +
  theme(legend.position = "bottom") +
  labs(title = "Wind Time Series, With and Without Outliers")
```



Answer: Limiting to just dates past January 2014, tsclean() removed far fewer outliers from the series. It seems to have removed one outlier from the solar data and no outliers from the wind data. This is because it is basing outliers relative to a mean that excludes the very low observations from before 2014.